

Отчет по backend и связанным частям проекта

Проект: Tour Uzbekistan

Дата формирования: 11.07.2026

Документ отражает текущее фактическое состояние backend на NestJS + Prisma + PostgreSQL, публичного API, seed-данных и уже добавленного foundation для административной панели.

1. Общая архитектура

Слой	Что используется
Backend	NestJS + TypeScript
ORM и БД	Prisma + PostgreSQL
Авторизация	JWT access/refresh tokens, bcrypt, guards, roles
Документация	Swagger на /api/docs
Контейнеризация	Docker Compose для PostgreSQL и backend
Frontend	Основной сайт в front, отдельная админка в admin на Vue 3

- Глобальный префикс API настроен как /api/v1.
- CORS включен через CORS_ORIGIN из .env.
- Глобально подключен ValidationPipe с whitelist, transform и forbidNonWhitelisted.
- ConfigModule подключен глобально, переменные читаются из .env.

2. Prisma schema и локализация

В схеме реализована единая стратегия локализации через отдельные translation-модели для ru, en и uz. Основные enum: Locale, UserRole, UserStatus, ContentStatus, TourType, Season, LeadStatus, BookingStatus и AudienceType.

- Основные сущности: User, Partner, Country, Tour, TourDay, TourImage, Service, HomeBanner, News, WhyCategory, WhyFact, Lead, Booking.
- Для локализации созданы отдельные таблицы переводов: UserTranslation, PartnerTranslation, CountryTranslation, TourTranslation, TourDayTranslation, TourImageTranslation, ServiceTranslation, HomeBannerTranslation, NewsTranslation, WhyCategoryTranslation, WhyFactTranslation, LeadTranslation, BookingTranslation.
- В контентных сущностях используются статусы публикации и сортировки там, где это нужно для frontend.
- В Tour и Booking предусмотрены поля для цены, транспорта, отелей, снапшотов и сопутствующих данных.

3. Seed и тестовые данные

- Seed очищает связанные таблицы в корректном порядке и заново наполняет базу данными.
- Создаются тестовые пользователи с ролями PARTNER, ADMIN и MANAGER.
- Для ADMIN и MANAGER пароли хешируются через bcrypt.

- Созданы данные для стран, туров, дней тура, изображений, услуг, новостей, блока 'Почему мы', заявок, баннеров и других сущностей, используемых frontend.
- Для всех ключевых сущностей seed заполняет переводы на ru, en и uz.

Тестовые административные учетные записи: superadmin@centrum-holidays.test / admin12345 и manager@centrum-holidays.test / manager12345.

4. Публичный backend API

Реализованные маршруты публичной части:

- GET /api/v1/health - health check.
- POST /api/v1/auth/register/partner - регистрация партнера.
- POST /api/v1/auth/login - логин пользователя.
- POST /api/v1/auth/refresh - обновление access/refresh token.
- GET /api/v1/home - данные для главной страницы из PostgreSQL.
- GET /api/v1/countries и GET /api/v1/countries/:slug - опубликованные страны с locale и 404.
- GET /api/v1/services и GET /api/v1/services/:slug - услуги с пагинацией и locale.
- GET /api/v1/why-us/categories и GET /api/v1/why-us/categories/:slug - категории и факты без N+1.
- GET /api/v1/news и GET /api/v1/news/:slug - новости с пагинацией, locale и сортировкой по publishedAt DESC.
- GET /api/v1/tours и GET /api/v1/tours/:slug - туры с фильтрами по стране, длительности, звездности, цене и поиску.
- POST /api/v1/leads - сохранение заявок с привязками к туру, стране и услуге.
- POST /api/v1/bookings и GET /api/v1/bookings/me - бронирования только для авторизованного PARTNER.

Особенность по турам: если пользователь не авторизован, backend не возвращает стоимость. Для booking сохраняется snapshot данных тура на момент создания.

5. Авторизация и роли

- Используются JWT access token и refresh token.
- Пароли хранятся в виде bcrypt hash.
- Поддерживаемые роли в UserRole: ADMIN, MANAGER, CUSTOMER, PARTNER.
- Для защиты маршрутов используются JwtAuthGuard и отдельный RolesGuard.
- Создан декоратор Roles(...roles) для role-based access control.
- PARTNER используется для partner auth и бронирований.
- ADMIN и MANAGER используются для административной части.

6. Административный foundation на backend

- Добавлен отдельный модуль backend/src/admin/admin.module.ts.
- Реализован admin auth API: POST /api/v1/admin/auth/login, POST /api/v1/admin/auth/refresh, GET /api/v1/admin/auth/me.
- Доступ к /api/v1/admin/* открыт только для ролей ADMIN и MANAGER.
- Реализован dashboard endpoint: GET /api/v1/admin/dashboard.
- Dashboard возвращает агрегированные счетчики по пользователям, партнерам, турам, услугам, новостям, заявкам и бронированиям, а также последние заявки и бронирования.

На текущий момент это готовый базовый слой для дальнейшего расширения admin CRUD, но не полная CMS по всем разделам из большого ТЗ.

7. Связанные части frontend

- Основной пользовательский frontend находится в папке front и не менялся архитектурно.
- Добавлена отдельная админ-панель в папке admin на Vue 3 + TypeScript + Vite + Vue Router + Pinia + Axios + Element Plus.
- В admin реализованы login, logout, route guards, bootstrap с проверкой /admin/auth/me и автоматическое обновление access token.
- Собран базовый layout: боковое меню, верхняя панель, breadcrumbs, имя пользователя и dashboard-страница.
- Admin frontend уже подключен к реальному backend API, а не к mock-данным.

8. Инфраструктура и запуск

- В backend/docker-compose.yml описаны сервисы postgres и backend.
- PostgreSQL по умолчанию пробрасывается на localhost:5432.
- Backend по умолчанию слушает localhost:3000.
- Swagger доступен по адресу /api/docs.
- Если локальный порт 5432 уже занят другим контейнером PostgreSQL, docker compose не поднимет собственный postgres без смены порта или остановки конфликтующего контейнера.

Команда	Назначение
npm run prisma:generate	Генерация Prisma Client
npm run prisma:validate	Проверка schema.prisma
npm run prisma:migrate:dev	Применение миграций в dev
npm run db:seed	Заполнение тестовыми данными
npm run build	Сборка backend

9. Проверка текущего состояния

На момент формирования этого отчета были повторно запущены локальные проверки сборки и валидации.

Проверка	Результат
backend: prisma validate	Успешно. Prisma schema валидна.
backend: npm run build	Успешно. Prisma Client сгенерирован, TypeScript сборка прошла без ошибок.
admin: npm run build	Успешно. Production build собран, есть только предупреждение Vite о крупных чанках.

10. Что уже закрыто и что остается

Уже реализовано:

- Базовый backend на NestJS + Prisma + PostgreSQL.
- Публичный API для home, countries, services, why-us, news, tours, leads и bookings.
- JWT auth, refresh token, bcrypt, роли CUSTOMER и PARTNER, затем расширение до ADMIN и MANAGER.
- Swagger, CORS, ValidationPipe, .env, Docker Compose.
- Prisma schema с переводами и seed с тестовыми данными.
- Foundation отдельной admin-панели и admin backend API.

Остается по большому admin ТЗ:

- Полный CRUD по всем сущностям и разделам CMS.
- Визуальное управление главной страницей, header, footer и настройками сайта.
- Разделы заявок и бронирований в админке с историей, комментариями, фильтрами, экспортом и назначением менеджеров.
- Управление пользователями и администраторами с разграничением прав для MANAGER.
- Медиа-менеджмент и загрузка файлов через админку.